

0.77. Plusieurs rythmes sans compteur : chaque

if (chaque(250)) { ... } : le bloc se fait toutes les 250 ms, sans rien arrêter et sans chronomètre à écrire. Le 0.16 en quatre lignes.

1. Ce qu'il faut comprendre

C'est le 0.16 (la lente et la rapide), **sans les chronomètres**. Au 0.16, chaque lettre avait sa variable qui comptait les images, et son `if` pour la remettre à zéro. Ici, une fonction de la console fait tout ça.

Ce qui est nouveau ici : `chaque(ms)`. Elle répond **1 (oui)** quand le temps est passé depuis la dernière fois, **0 (non)** le reste du temps. `if (chaque(1000)) { ... }` : le bloc se fait **une fois par seconde**. `if (chaque(250)) { ... }` : **quatre fois par seconde**.

Elle n'arrête rien, contrairement à `attendre()` : le reste de la boucle continue de tourner à chaque image. On peut donc avoir **plusieurs rythmes** dans le même programme.

Chaque `chaque` a son propre chronomètre : la console les distingue toute seule, par leur place dans le programme. Les deux lignes ci-dessous ne se gênent pas. On peut en écrire jusqu'à huit.

`(lente + 1) % sizeof(ALPHABET)` passe à la lettre suivante, et revient à 0 après la 26e (le Z) : le `%` du 0.9.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <reste>` → [0.8.4. #include <reste> — a % b](#)
- `#include <chaque>` → [0.29.3. La fonction chaque\(\) — toutes les n millisecondes](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1065 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage          55 o ← la console, à l'allumage — la mise en route de la console
chaque_minuteur() 124 o ← main() — une fonction fournie par le langage
main()             88 o ← démarrage — ta fonction
VBlank             8 o ← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage      28 o ← image() — attend l'image suivante
AttendreVBlank     16 o ← démarrage — attend que l'écran soit libre
Reste              10 o ← main() — le reste d'une division
CopierTuiles       18 o ← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte       14 o ← démarrage — vide les deux cartes de l'écran
Tuiles             704 o ← CopierTuiles — la police entière
Lettres gravées : toutes (poser() reçoit un numéro calculé)
Pas gravé (jamais appelé) : AvancerAirs, AttendreAcces, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireTexte,
EcrireNombre, Diviser, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard,
EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes
Fonctions du langage appelées :
```

```

poser()          × 2    24 o sur place, dans main()
image()          × 1     3 o sur place, dans main() + AttendreImage (28 o)
images()         × 1     2 o sur place, dans chaque_minuteur()
Tes fonctions appelées :
chaque_minuteur() × 2    21 o sur place, dans main()

```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```

#include <reste>    // a % b, sauf par 1, 2, 4, 8, 16... écrits en clair
#include <chaque>   // répond 1 toutes les n millisecondes
#include <ALPHABET> // les lettres de la police : ALPHABET[0] est le A
#include <poser>    // pose une tuile sur une case du fond

```

2. UNE VARIABLE — LENTE

```
uint8_t lente = 0;    // la lettre de la lente : 0 = A ... 25 = Z
```

3. UNE VARIABLE — RAPIDE

```
uint8_t rapide = 0;   // la lettre de la rapide
```

4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```

int main() {
    while (true) {
        image();

        // La fonction, morceau par morceau :
        //
        //   if (chaque(1000)) { ... }
        //       |           |
        //       |           +-- toutes les 1000 millisecondes (1 seconde)
        //       +----- réponde 1 quand c'est l'heure, 0 sinon : le bloc
        //               ne se fait qu'aux bons moments, rien n'est arrêté
        if (chaque(1000)) {
            lente = (lente + 1) % sizeof(ALPHABET);    // la lente : 1 fois par seconde
        }
        if (chaque(250)) {
            rapide = (rapide + 1) % sizeof(ALPHABET);  // la rapide : 4 fois par seconde
        }

        poser(0, 0, ALPHABET[lente]);    // ligne 0 : la lente
        poser(0, 2, ALPHABET[rapide]);    // ligne 2 : la rapide
    }
}

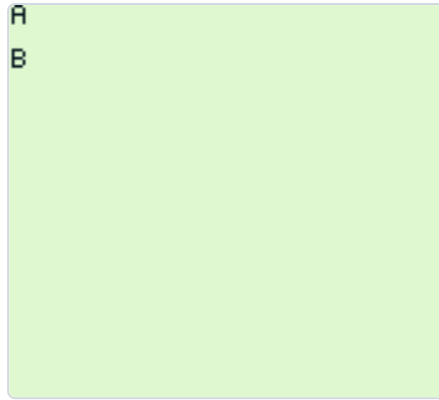
```

3. Ce qui se passe, étape par étape

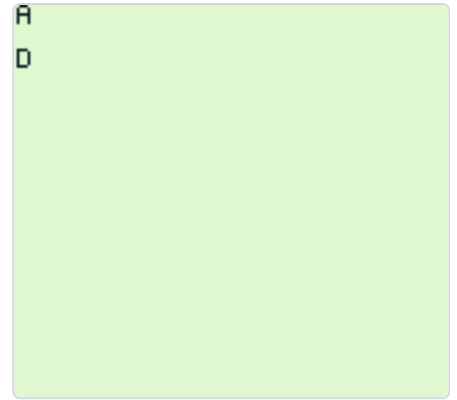
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



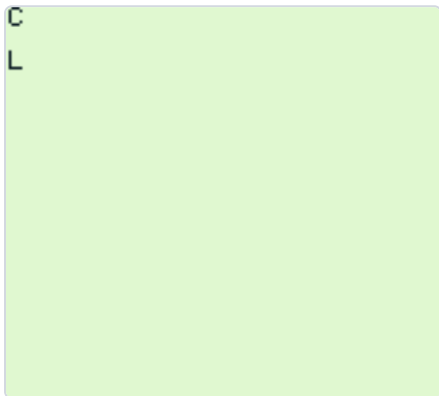
la toute première image — la console vient de démarrer — image 1



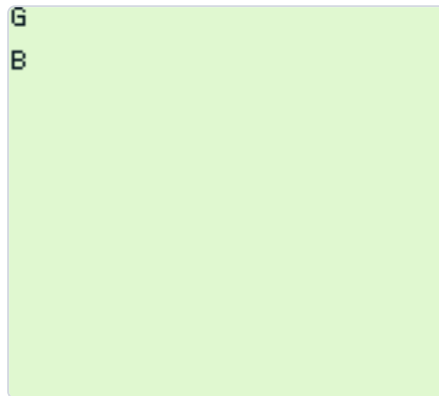
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Comme au 0.16 : en haut, une lettre change chaque seconde ; en dessous, une autre quatre fois plus vite.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ en 4 secondes, la lente a changé 4 fois (4)

✓ la rapide 16 fois (16)

Ensuite : **0.78. Le rebond : une vitesse qui change de signe** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++